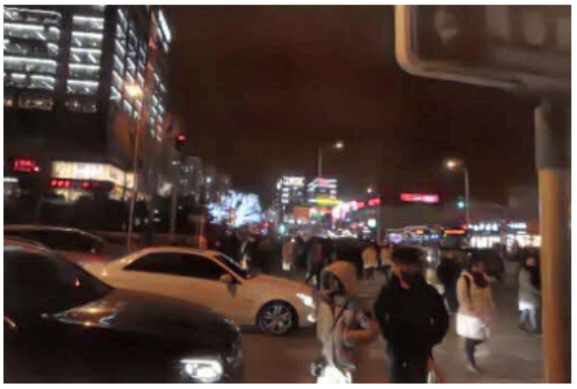
1. You are provided with a clean grayscale image as your ground truth. Apply three different types of noise Gaussian, Salt & Pepper, and Speckle to create three separate degraded versions of the image. Using appropriate denoising filters for each noise type, restore all three images as closely as possible to the original.

2.You are provided with a dull, low-contrast grayscale image as your input. Apply the intensity scaling techniques of your choice (linear scaling / gamma correction / log transformation / contrast stretching to enhance the brightness and sharpness of the image. Compare each enhanced output against the original using visual inspection transformation to demonstrate the improvement.

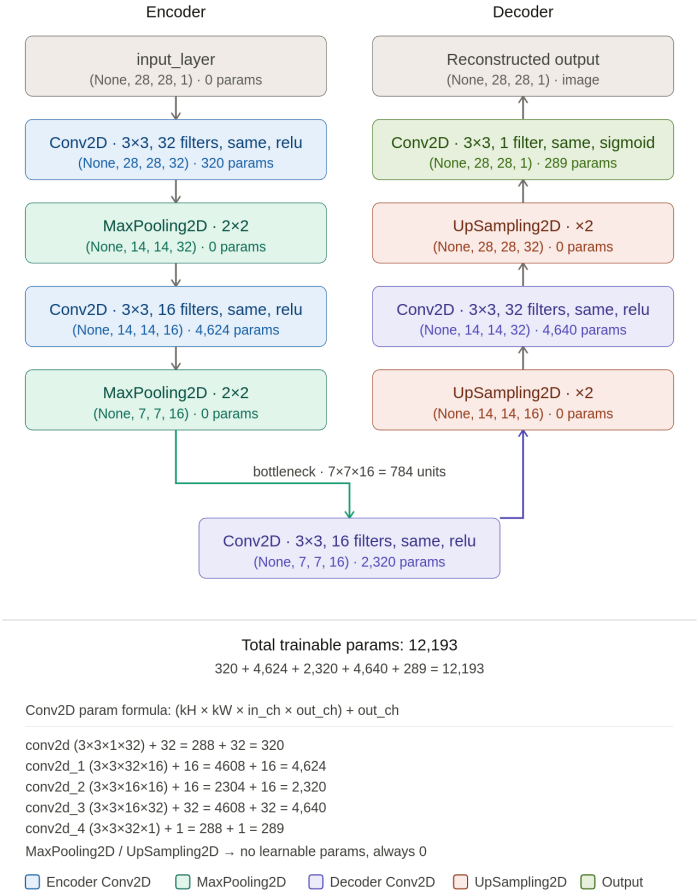
Low Light Image

Enhanced Image



Question 3 Implement the encoder-decoder architecture shown in the diagram above using the Keras Functional API. Your implementation will be evaluated by running model.summary() the total

trainable parameter count must exactly match 12,193.



4. You are provided with an image and a randomly generated filter kernel (given at runtime) without using any deep learning library such as TensorFlow or PyTorch. Implement the convolution operation from scratch using only NumPy, applying the kernel over the image to preserve the output dimensions. Display the original and convolved output image side by side and report the kernel values used and the effect it produced on the image.

5. Your goal is to match the ground truth output as closely as possible by experimenting with different filters, thresholds, and post-processing steps, and report your approach and the final visual result.

QUESTION 6 : CNN Adaptation

You are given a working CNN for BINARY classification of cats vs dogs.
The dataset folder structure is:

```
binary_dataset/
├── train/
│   ├── cat/     ← 80 images (BGR format, loaded via OpenCV)
│   └── dog/     ← 80 images (BGR format, loaded via OpenCV)
└── test/
    ├── cat/     ← 20 images
    └── dog/     ← 20 images
```

You are also given a NEW dataset for MULTI-CLASS classification:

```
flower_dataset/
├── train/
│   ├── rose/       ← 60 images  (BGR format — OpenCV loaded)
│   ├── tulip/      ← 60 images
│   ├── daisy/      ← 60 images
│   ├── sunflower/  ← 60 images
│   └── dandelion/  ← 60 images
└── test/
    ├── rose/       ← 15 images
    ├── tulip/      ← 15 images
    ├── daisy/      ← 15 images
    ├── sunflower/  ← 15 images
    └── dandelion/  ← 15 images
```

NOTE: This is a deliberate bug you must fix.

TASKS

Task 6a - Fix BGR → RGB
The loader reads images in BGR. Fix this BEFORE feeding to the model

Task 6b - Adapt the Dataset Loader for Multi-class
- Automatically detect all class folders
- Assign integer labels 0, 1, 2, ... for each class alphabetically

Task 6c - Modify the Model
Change the model output layer and compilation for 5-class classification:

STARTER CODE (Binary Classification)

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import cv2
import os
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report

def load_dataset(folder, img_size=(64, 64)):
    images, labels = [], []

    for label, cls in enumerate(['cat', 'dog']):                #
        cls_folder = os.path.join(folder, cls)
        for fname in os.listdir(cls_folder):
            if not fname.lower().endswith(('.jpg', '.jpeg', '.png')):
                continue
            img = cv2.imread(os.path.join(cls_folder, fname))     #
            img = cv2.resize(img, img_size)
            img = img / 255.0                                #
            images.append(img)
            labels.append(label)

    return np.array(images), np.array(labels)                    #


# Load binary dataset
X_train, y_train = load_dataset('binary_dataset/train')
X_test,  y_test  = load_dataset('binary_dataset/test')

print(f"Train shape: {X_train.shape}, Labels: {y_train.shape}")
print(f"Test shape : {X_test.shape},  Labels: {y_test.shape}")
model = models.Sequential([
    layers.Input(shape=(64, 64, 3)),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
```

```
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=10,
    validation_data=(X_test, y_test),
    verbose=1
)
```

Images : https://drive.google.com/drive/folders/1JSI1DIGh42Ooyhp-oHyFpQTa3lmQPvFP?usp=sharing